

GUILHERME GIACOMINI TEIXEIRA

AULA PRÁTICA - TRADUTORES E COMPILADORES
PROCESSAMENTO DE LINGUAGEM NATURAL

BALNEÁRIO CAMBORIÚ - SC

2026

GUILHERME GIACOMINI TEIXEIRA

AULA PRÁTICA - TRADUTORES E COMPILADORES

Trabalho de avaliação da disciplina Processamento de Linguagem Natural apresentado como requisito para a obtenção da média no curso Tecnólogo em Ciência de Dados.

Professor: ANDERSON INACIO SALATA DE ABREU

BALNEÁRIO CAMBORIÚ - SC

2026

SUMÁRIO

1 INTRODUÇÃO	4
2 DESENVOLVIMENTO	5
3 RESULTADOS	8
4 CONCLUSÃO	9
5 REFERÊNCIAS	10

1 INTRODUÇÃO

O Processamento de Linguagem Natural (NLP) é uma subárea da inteligência artificial que visa aproximar a computação estruturada da linguagem humana desestruturada. Para que algoritmos possam computar inferências estatísticas ou semânticas a partir de textos em linguagem natural, o input (texto bruto) necessita passar por um pipeline de normalização e transformação vetorial.

Este relatório documenta a execução de um experimento prático computacional cujo objetivo foi aplicar técnicas elementares de pré-processamento textual. Utilizando a linguagem Python com o auxílio das bibliotecas NLTK e SpaCy, o projeto implementou rotinas para marcação morfosintática (Part-of-Speech Tagging), normalização (remoção de ruídos, extração de raízes e lematização) e segmentação de tokens.

O presente documento expõe a análise técnica da prova de conceito, os recortes das execuções lógicas e a validação dos conceitos fundamentais de NLP requeridos pela proposta acadêmica.

2 DESENVOLVIMENTO

2.1 Análise da Prova de Conceito Computacional

O experimento foi estruturado como um pipeline sequencial aplicado sobre um texto de entrada padrão, desenvolvido em ambiente Python isolado. As respostas comportamentais das bibliotecas NLTK e SpaCy em cada etapa foram as seguintes:

A) Segmentação de Texto (Sentence Splitting e Tokenization)

A divisão do texto bruto em unidades processáveis foi realizada no nível de sentenças (via identificação de pontuação terminativa) e no nível de palavras (tokens). O algoritmo de tokenização demonstrou separar com precisão as unidades lexicais, mantendo a coerência em relação aos sinais de pontuação subjacentes. A segmentação computacional é o passo base, pois transforma uma string de caracteres linear em um vetor iterável que alimentará os modelos seguintes.

B) Marcação Morfossintática (POS Tagging)

Após a tokenização, o modelo pré-treinado do SpaCy para a língua portuguesa (pt_core_news_sm) foi instanciado sobre o vetor. O algoritmo realizou a classificação sintática de cada token, gerando mapeamentos no formato de chave e valor. Termos estruturais como Inteligência foram mapeados corretamente para substantivos, enquanto verbos transitivos receberam a tag correta. Essa etapa garantiu a extração da função lógica da palavra na estrutura da frase, superando a mera análise literal.

C) Normalização Textual (Stopwords, Stemming e Lematização)

Para a normalização, o pipeline foi dividido em três fluxos paralelos:

1. Remoção de Stopwords: A aplicação do dicionário padrão do NLTK filtrou as palavras estruturais sem valor semântico independente (como preposições e

artigos).

2. Stemming: O algoritmo SnowballStemmer operou através de corte heurístico de prefixos e sufixos. Identificou-se que a técnica reduz os tokens de maneira agressiva ao seu radical.

3. Lematização: Executada via motor do SpaCy, a lematização preservou a integridade linguística, transformando conjugações e plurais nas suas entradas normativas de dicionário.

2.2 Resolução de Conceitos Específicos

1) Qual é a função da marcação morfossintática em NLP?

A marcação morfossintática (POS Tagging) tem a função de assinalar a classe gramatical de cada token (substantivo, adjetivo, verbo, advérbio) dentro do seu contexto sintático. Em NLP, isso é fundamental para a desambiguação semântica de sistemas matemáticos. Um classificador precisa distinguir algoritmicamente se a palavra canto foi usada como um substantivo (o canto da sala) ou como a flexão do verbo cantar. Sem essa marcação estrutural, a extração de sentido em análises mais profundas fica severamente comprometida.

2) Qual é a diferença entre stemming e lematização?

Ambas são técnicas de redução morfológica para agrupar variações de uma mesma palavra, mas diferem no rigor algorítmico. O stemming é heurístico e reativo; ele aplica regras estáticas de corte de sufixos para chegar a um radical comum, muitas vezes gerando radicais que sequer existem como palavras válidas (ex: estudante e estudar viram estud). A lematização, por sua vez, realiza uma análise morfológica sistêmica em tempo de execução, consultando um vocabulário interno para reverter a palavra à sua raiz de dicionário correta (o lema). Por exemplo, a lematização identifica que fui e ir derivam do mesmo lema base, algo impossível de ser feito por corte de sufixos via stemming.

3) Quais problemas podem surgir durante a segmentação de textos?

A etapa de segmentação sofre com ambiguidades lógicas em casos limite.

Algoritmos ingênuos que utilizam o caractere de ponto como sinalizadores restritos de final de sentença tendem a falhar ou dividir strings incorretamente frente a abreviações (Sr., EUA) ou números decimais formatados (3.14). Além disso, hifenizações, URLs, hashtags e menções geram complexidade; seccioná-los puramente por caracteres de espaçamento corrompe o valor da informação semântica original contida neles. O uso de tokens padronizados e Expressões Regulares (Regex) bem definidas resolve grande parte desses impasses.

3 RESULTADOS

A implementação computacional permitiu observar as implicações diretas da escolha do fluxo de processamento de texto. Enquanto técnicas primitivas como a contagem frequencial sofrem com alto ruído estatístico quando expostas a stopwords, sua filtragem imediata gerou matrizes de dados (vetores) reduzidos e adensados semanticamente.

Computacionalmente, constatou-se que a lematização consome maior recurso de processamento por exigir dicionários acoplados no motor, mas devolve dados íntegros e compatíveis linguisticamente. O stemming, por ser mais barato do ponto de vista algorítmico, tem validade em projetos onde o peso semântico exato da palavra é menos crítico do que a velocidade de indexação e recuperação no banco de dados (como arquiteturas básicas de motores de busca).

A marcação morfossintática demonstrou alta coerência, possibilitando que um algoritmo isolasse automaticamente apenas verbos e substantivos do texto, caso o desenvolvedor desejasse extrair somente o núcleo da ação das sentenças.

4 CONCLUSÃO

O experimento validou a importância das etapas sequenciais no processamento de linguagem natural. Algoritmos mais limpos na sua entrada produzem cálculos determinísticos mais precisos na saída. A aplicação de tokenização cuidadosa e normalização inteligente reduz agressivamente a dimensionalidade espacial do processamento, evitando gasto de recursos computacionais com informações gramaticais inúteis para os modelos.

Fica claro que não existe uma técnica de redução superior absoluta. Stemming e Lematização, embora busquem o mesmo objetivo, atendem a restrições arquiteturais diferentes: o primeiro prioriza a velocidade de execução; o segundo, o rigor de contexto e conservação semântica. A escolha do método deve sempre subordinar-se aos requisitos de desempenho e memória exigidos pela regra de negócio do projeto em questão.

5 REFERÊNCIAS

ALENCAR, Ana Catarina de. Inteligência Artificial, Ética e Direito: Guia Prático para Entender o Novo Mundo. São Paulo: Expressa, 2022.

Big Data Analytics. ISSN 2058-6345.

GABRIEL, Martha. Inteligência Artificial: Do Zero ao Metaverso. Rio de Janeiro: Atlas, 2022.

International Journal of Advanced Research in Computer Science. ISSN 0976-5697.

KAUFMAN, Dora. Desmistificando a inteligência artificial. São Paulo: Autêntica Editora, 2022.

MORAIS, Izabelly Soares de...[et al]. Introdução a Big Data e Internet das Coisas (IoT). Porto Alegre: SAGAH, 2018.

PRADO, Magaly. Fake News e Inteligência Artificial: O poder dos algoritmos na guerra da desinformação. São Paulo: Grupo Almedina, 2022.

SILVA, Fernanda Rosa da... [et al.]. Cloud computing. Porto Alegre: SAGAH, 2020.